SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Faculty of Computing

Year 2 Semester 1 (2026)

IT2011 - Artificial Intelligence and Machine Learning

Lab Sheet 03

Supervised Learning I

Learning Objectives

1. Implement and evaluate linear regression models using scikit-learn to predict a continuous outcome.
2. Visualize model predictions and feature relationships to understand linear regression's predictive behavior.

Introduction

This task focuses on applying linear regression, a supervised learning method for predicting continuous outcomes, using the diabetes dataset. You will preprocess data, build and evaluate a linear regression model, and visualize predictions to understand how features influence the target. The tasks emphasize step-by-step implementation, visualization, and interpretation of results, aligning with model assumptions, evaluation metrics (MSE, RMSE, R^2). By the end of this lab, you will gain hands-on experience in applying Simple Linear Regression and Multiple Linear Regression models and understanding their strengths and limitations.

Task 1 - Linear Regression – (01 hour)

Linear regression is a fundamental supervised learning algorithm for predicting continuous outcomes, assuming a linear relationship between input features (independent variables) and the target variable (dependent variable). It can be simple (one feature) or multiple (several features), and it minimizes the sum of squared errors (ordinary least squares) to find the best-fit line.

In this task, you will use the diabetes dataset from scikit-learn (Dataset), which contains 10 features (e.g., age, BMI, blood pressure) for 442 patients and a target measuring disease progression after one year. The goal is to predict disease progression. This task builds your understanding of how correlation influences model fit and how to interpret metrics like mean squared error (MSE) and R^2 (coefficient of determination), which indicates the proportion of variance explained by the model.

Follow these steps in a new notebook named **"Task1_LinearRegression.ipynb".**

Step 1: Import Necessary Libraries

Import necessary Python libraries for data handling, visualization, preprocessing, and linear regression modeling.

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Step 2: Load the Dataset
Load the raw diabetes dataset from the provided URL as a tab-delimited file.

```
# Load the raw diabetes dataset from the URL
url = "https://www4.stat.ncsu.edu/~boos/var.select/diabetes.tab.txt"
data = pd.read_csv(url, sep='\t')  # Tab-delimited file

# Display column headers
print("Original column headers:")
print(list(data.columns))
```

Step 3: Rename Column Headers
Inspect the original column headers and rename them to more descriptive names for clarity (e.g. Y to DiseaseProgression).

```
# Rename headers for clarity
new_columns = {
    'AGE': 'Age', 'SEX': 'Sex', 'BMI': 'BMI', 'BP': 'BloodPressure',
    'S1': 'SerumCholesterol','S2': 'SerumLDL','S3': 'SerumHDL','S4': 'SerumTCH',
    'S5': 'SerumLTG','S6': 'SerumGlucose','Y': 'DiseaseProgression'}

data.rename(columns=new_columns, inplace=True)

# Display top 5 rows
print("\nTop 5 rows of dataset")
print(data.head())
```

Step 4: Separate the dataset in to Features and Target and convert to Pandas DataFrame

Split the dataset into features (X) and target (y, DiseaseProgression), and ensure features are stored as a pandas DataFrame for easy manipulation.

```
# Separate features (X) and target (y)
X = data.drop(columns=['DiseaseProgression'])  # Features
y = data['DiseaseProgression']  # Target

# Convert features to pandas DataFrame
pd_X = pd.DataFrame(X)

# Convert target to pandas Series for display
pd_Y = pd.Series(y)

# Display top 5 rows
print("\nTop 5 rows of Features")
print(pd_X.head(5))
print("\nTop 5 rows of Target")
print(pd_Y.head(5))
```

Step 5: Preprocess the Data

Check for missing values, handle the categorical feature (Sex), and scale numerical features to standardize them (mean=0, variance=1) for consistency and interpretability

```
# Check missing values
print("Missing values:", data.isnull().sum())
if data.isnull().sum().sum() == 0:
    print("No missing values found.")
else:
    data = data.dropna()
    X = data.drop(columns=['DiseaseProgression'])
    y = data['DiseaseProgression']

# Scale numerical features
numerical_cols = [col for col in X.columns if col != 'Sex']
scaler = StandardScaler()
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])
print("\nFeatures after scaling (top 5 rows):")
display(X.head())
```

Sex is the only categorical column. However, it is already in numerical representation (1=male, 2=female). Therefore, no columns to encode.

Step 6: Compute and Visualize Correlation Matrix

Calculate correlations between features and the target (DiseaseProgression) to identify strong and weak linear relationships, aiding feature selection for regression.

```
# Combine features and target for correlation
data_temp = X.copy()
data_temp['DiseaseProgression'] = y
corr_matrix = data_temp.corr()

# Display the full correlation matrix
print("\nFull Correlation Matrix:")
display(corr_matrix)

# Generate heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', square=True)
plt.title('Correlation Heatmap of Features and DiseaseProgression')
plt.show()
```

Step 7: Visualize Feature Relationships

Create scatter plots for the features with top 3 highest correlations to the target to visualize potential linear relationships.

```
top_features = corr_matrix['DiseaseProgression'].abs().sort_values(ascending=False).index[1:4]
fig, axes = plt.subplots(1, 3, figsize=(18, 4))
for i, feature in enumerate(top_features):
    ax = axes[i]
    ax.scatter(X_test[feature], y_test, color='blue', label='Data')
    ax.set_xlabel(feature)
    ax.set_ylabel('DiseaseProgression')
    ax.set_title(f'{feature} vs DiseaseProgression')
    ax.legend()
plt.tight_layout()
plt.show()
```

Step 8: Split Data into Training and Testing Sets

Divide the dataset into 80% training and 20% testing sets to train and evaluate the model on unseen data.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training samples: {len(X_train)}, Testing samples: {len(X_test)}")
```

Step 9: Train a Linear Regression Model

Fit a multiple linear regression model using all features to predict DiseaseProgression and display coefficients and intercept.

```
model = LinearRegression()
model.fit(X_train, y_train)
display("Model coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

Step 10: Evaluate Model Performance

Predict on the test set and compute metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE) to measure prediction error and R^2 (coefficient of determination) to assess how well the model explains variance.

```
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R² Score: {r2:.2f}")
```

Step 11: Visualize Predictions

Plot true vs. predicted DiseaseProgression values for the test set to visually assess model fit, with a best fit line for perfect predictions.

```
plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, color='blue', label='Predicted vs True')
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r', label='Perfect Prediction')
plt.xlabel('True DiseaseProgression')
plt.ylabel('Predicted DiseaseProgression')
plt.title('True vs Predicted Disease Progression (Test Set)')
plt.legend()
plt.show()
```

Step 12: Experiment with Simple Linear Regression

Train a simple linear regression model using only the highest-correlated feature, compute MSE, RMSE, and R^2, and compare with the multiple regression model to explore lecture concepts (simple vs. multiple regression).

```
# Simple Linear Regression with Highest Correlated Feature
highest_corr = corr_matrix['DiseaseProgression'].abs().sort_values(ascending=False).index[1]
X_train_single = X_train[[highest_corr]]
X_test_single = X_test[[highest_corr]]
simple_model = LinearRegression()
simple_model.fit(X_train_single, y_train)
y_pred_single = simple_model.predict(X_test_single)
mse_single = mean_squared_error(y_test, y_pred_single)
rmse_single = np.sqrt(mse_single)
r2_single = r2_score(y_test, y_pred_single)
print(f"Simple Regression (using {highest_corr}):")
print(f"MSE: {mse_single:.2f}, R²: {r2_single:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_single:.2f}")
print(f"R² Score: {r2:.2f}")
```

Step 13: Reflect on Correlation and Linear Regression

Add a brief note in the notebook explaining the relationship between correlation and linear regression performance. Explains why multiple regression (more features) outperforms simple regression.